CS110 (Principles of CS)
RECITATION 6
Python – Sequential Programming

RECITATION INSTRUCTOR – Dharm Patel
SECTION – 6

Agenda

01 **IDE setup & exam Q&A**
Where we write and run code

02 **Input & casting**
The Str → int / float pattern

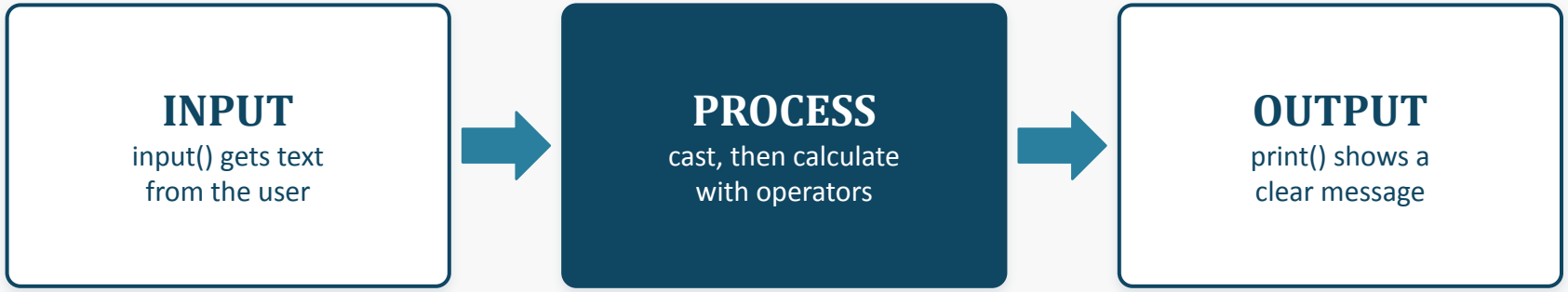
03 **Operators**
/ // % * ** and comparisons

04 **Programs 1–5**
Temperature, MPG, capacity, holiday, area

05 **Bonus programs**
Even/odd, accumulating values

06 **Math & printing**
ceil, floor, round, \n, \t, time math

What Is Sequential Programming?



Today's rules

- Code runs line by line, top to bottom — order matters!
- No `if` statements today. We use Boolean expressions instead.
- We write every program together, so call out the next line before I type it.

The Input & Casting Pattern

Python

```
# Step 1: input() ALWAYS returns a string
valueStr = input("Enter a value: ")

# Step 2: cast it, drop the 'Str'
value = float(valueStr)
```

Why cast?

- `"5" + "3"` → `"53"` (text joins)
- `5 + 3` → `8` (math works)
- We can only calculate with numbers, not text.

`int()`
Whole numbers
people, days, items

`float()`
Decimal numbers
temperature, miles, gallons

`str`
Text
what input() gives you

Arithmetic Operators: / vs // vs %

`7 / 2`

3.5

Division
always a decimal

`7 // 2`

3

Floor division
drops the decimal

`7 % 2`

1

Modulo
the remainder

`7 * 2`

14

Multiply
times

`7 ** 2`

49

Exponent
7 to the power 2

Python

```
a = 10
b = 3
print(a / b, a // b, a % b)
```

Sample run

```
3.3333333333333335 3 1
```

Comparison Operators: True or False

Operator	Meaning	Example	Result
`==`	equal to	`5 == 3`	`False`
`!=`	not equal to	`5 != 3`	`True`
`>`	greater than	`5 > 3`	`True`
`<`	less than	`5 < 3`	`False`
`>=`	greater or equal	`5 >= 5`	`True`
`<=`	less or equal	`5 <= 3`	`False`

Key idea

- A comparison asks a yes/no question.
- Python answers with `True` or `False`.
- That answer is a Boolean value.
- Don't mix up `=` (assign) and `==` (compare)!

Program 1: Fahrenheit to Celsius

Formula: `C = (F − 32) × 5 / 9`

Input is text, so cast it to float before the math.

Python

```
# degreesC = (degreesF - 32) * 5/9
degreesFStr = input("Enter degrees Fahrenheit: ")
degreesF = float(degreesFStr)
degreesC = (degreesF - 32) * 5 / 9
print("")
print("Celsius degrees =", round(degreesC, 2))
```

Explanation

- `degreesFStr` holds raw text
- `float()` turns it into a number
- Parentheses: subtract 32 first
- `round(x, 2)` → 2 decimals
- `print("")` → blank line

Sample run

```
Enter degrees Fahrenheit: 100

Celsius degrees = 37.78
```

Program 2: MPG Calculator

Formula: `MPG = miles ÷ gallons`

Two inputs, two casts, one division, one nice message.

Python

```
milesStr = input("How many miles did you drive? ")
miles = float(milesStr)
galsUsedStr = input("How many gallons did you use? ")
galsUsed = float(galsUsedStr)
mpg = miles / galsUsed
print("\nYou got", round(mpg, 2), "miles to the gallon")
```

Explanation

- Same Str ⟶ cast pattern, twice
- `/` gives a decimal answer
- `\n` skips a line before printing
- Commas in print() add spaces for us

Sample run

```
How many miles did you drive? 310
How many gallons did you use? 11.5

You got 26.96 miles to the gallon
```

Program 3: Venue Capacity (No IF!)

Is the event within the safe limit?

Store the Boolean expression in a variable, then print it.

Python

```
# Venue capacity
capacityStr = input("Enter the capacity of this venue: ")
capacity = int(capacityStr)  # people are whole numbers
guestsStr = input("Enter the number of guests: ")
guests = int(guestsStr)
withinLimits = guests <= capacity
print("Within limits?", withinLimits)
```

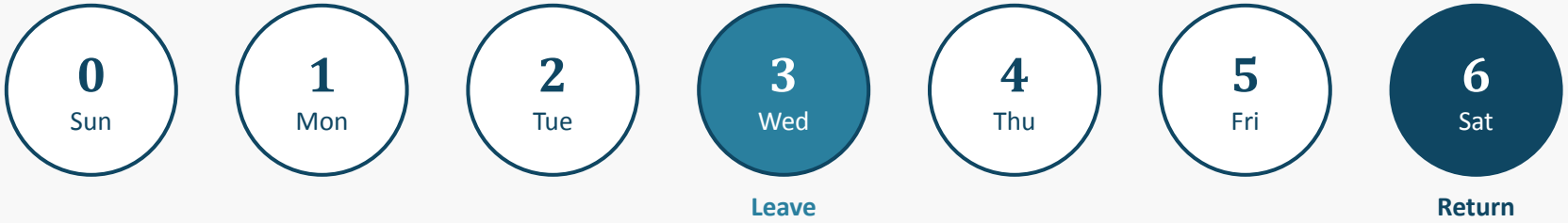
Explanation

- `int`, not float: no half-people!
- `guests <= capacity` evaluates to True or False
- `withinLimits` stores that Boolean
- No `if` needed

Sample run

```
Enter the capacity of this venue: 150
Enter the number of guests: 120
Within limits? True
```

Modulo (%): Numbers That Wrap Around



`3 + 10`
start day + nights

`= 13`
past the end of the week

`13 % 7 = 6`
wrap around → Saturday

`% 7` always gives a result from 0 to 6, so the answer always lands on a real day. Try: `13 % 7` → 6, `14 % 7` → 0

Program 4: Holiday Return Day

Days are numbered 0 (Sun) to 6 (Sat).

Add the stay length, then use % 7 to wrap around the week.

Python

```
# This problem MAY HAVE BEEN DONE in class
startDayStr = input("What day number do you leave? ")
startDay = int(startDayStr)
numDaysStr = input("How many days will you stay? ")
numDays = int(numDaysStr)
returnDay = (startDay + numDays) % 7
print("You will return on day", returnDay)
```

Explanation

- Both inputs cast to int
- Add start day + length of stay
- `% 7` keeps the answer between 0 and 6
- Parentheses: add first, then mod

Sample run

```
What day number do you leave? 3
How many days will you stay? 10
You will return on day 6
```

Program 5: Rectangle Area

Formula: `Area = length × width`

A shortcut: input + cast in one line, math inside print().

Python

```
# rectangle
length = float(input("Enter rectangle length: "))
width = float(input("Enter rectangle width: "))
# calculating inline in the print statement
print("The area of this rectangle is:", length * width)
```

Explanation

- `input()` runs first, then `float()`
- Casting and input happen on the same line
- `length * width` is calculated right inside print()
- Shorter code, same result

Sample run

```
Enter rectangle length: 5.5
Enter rectangle width: 4
The area of this rectangle is: 22.0
```

Bonus 6: Even or Odd (No IF!)

Even numbers have remainder 0 when divided by 2.

Combine % with == to get a Boolean answer.

Python

```
# Another Boolean expression and MOD
num = int(input("Enter an integer: "))
even = num % 2 == 0
print("IS IT EVEN?", even)
```

Explanation

- `num % 2` → 0 (even) or 1 (odd)
- `== 0` checks if the remainder is zero
- `%` runs before `==`
- `even` holds True or False

Sample run

```
Enter an integer: 7
IS IT EVEN? False
```

Bonus 7: Accumulating Values

Python

```
# Accumulating values
total = 0
num1 = int(input("Enter num1: "))
total = total + num1
num2 = int(input("Enter num2: "))
total = total + num2
num3 = int(input("Enter num3: "))
total = total + num3
print("The total is:", total)
average = total / 3
print("The average is:", round(average, 2))
```

Trace it! (inputs: 4, 7, 10)

Step	total
`total = 0`	**0**
`+ num1 (4)`	**4**
`+ num2 (7)`	**11**
`+ num3 (10)`	**21**
`average = 21 / 3`	**7.0**

Start at 0, then **`total = total + num`** adds each new value to the running sum.

Math Functions: ceil() and floor()

Python

```
import math
num = float(input("Input a float: "))
print("ceil: ", math.ceil(num))
print("floor:", math.floor(num))
```

Sample run

```
Input a float: 3.2
ceil:  4
floor: 3
```

num	math.ceil(num)	math.floor(num)
3.2	4	3
3.7	4	3
5.0	5	5
-2.3	-2	-3

Remember

- `import math` goes at the top
- `ceil` → rounds UP (ceiling)
- `floor` → rounds DOWN (floor)

Printing & Rounding Techniques

Code	What it does	Result
`print("")`	Prints a blank line	`(empty line)`
`print("\nTitle")`	Skips a line, then prints	`(empty line)  Title`
`print("\tTitle Indented")`	Indents with a tab	`Title Indented`
`round(3.4567, 2)`	Round to 2 decimal places	`3.46`
`round(3.4567)`	Round to a whole number	`4`

`\n` = new line `\t` = tab

Escape characters start with a backslash `\`

Challenge: Seconds to Hours, Minutes & Seconds

60 seconds = 1 minute 3600 seconds = 1 hour

Uses // and % together, still fully sequential.

Python

```
totalSecStr = input("Enter total seconds: ")
totalSec = int(totalSecStr)
hours = totalSec // 3600
leftover = totalSec % 3600
minutes = leftover // 60
seconds = leftover % 60
print("\t", hours, "h", minutes, "m", seconds, "s")
```

Explanation

- `// 3600` → full hours
- `% 3600` → seconds left over
- `// 60` → full minutes
- `% 60` → final seconds

Sample run

```
Enter total seconds: 4000
     1 h 6 m 40 s
```

Thank you

Questions?

Dharm Patel | Section 6